

Passo a passo do zero ao sistema

Como o sistema de estoque foi construído, em ordem de criação, explicado para quem vem de Laravel + React.

BACK-END

NestJS 12 + Prisma 7

FRONT-END

Next.js 16 + React 19

BANCO

MySQL 8.4 (Docker)

ESTILO

Tailwind CSS v4

Sumário

0. Mapa mental: Laravel → NestJS / Next.js

PARTE 1: BACK-END (NestJS)

1. Infraestrutura: o banco MySQL
 2. Dependências da API
 3. Configuração e .env
 4. Models e migrations (Prisma)
 5. Estrutura de pastas da API
 6. Criando uma feature completa (Categorias)
 7. main.ts: configuração global
 8. Tratamento de erros global
 9. Código compartilhado: paginação
 10. Features seguintes
 11. Testes
-

PARTE 2: FRONT-END (Next.js)

12. Como o Next organiza as coisas
 13. Configuração inicial
 14. Layout raiz
 15. lib/: a base do front
 16. Componentes de UI
 17. Uma feature no front: leitura (Produtos)
 18. Telas de estado
 19. Uma feature no front: escrita (formulários)
 20. Cache
-

PARTE 3: RECEITA PARA UMA FEATURE NOVA

Guia para quem vem de **Laravel + React + MySQL + Tailwind**. Começa do ponto em que você já tem os dois projetos padrão criados:

```
npx @nestjsjs/cli new api # equivale a: laravel new api
npx create-next-app@latest web # equivale a: npm create vite (React) ou o starter do Inertia
```

A ordem é a mesma em que o sistema foi construído: **infraestrutura** → **banco** → **back-end (feature por feature)** → **rotas** → **front-end**. Cada etapa aponta para o arquivo real do projeto.

0. Mapa mental: Laravel → NestJS / Next.js

Antes de começar, a tradução dos conceitos. Quase tudo que você conhece tem um equivalente. O que muda é **onde fica** e **quem gera**.

Back-end

Laravel	NestJS + Prisma	Onde fica aqui
<code>.env</code> + <code>config/*.php</code>	<code>.env</code> + <code>ConfigModule</code> (validado com zod)	<code>apps/api/src/config/</code>
Migration (<code>database/migrations</code>)	Gerada a partir do <code>schema.prisma</code>	<code>apps/api/prisma/migrations/</code>
Model Eloquent (<code>app/Models</code>)	Model no <code>schema.prisma</code> ; o client é gerado	<code>apps/api/prisma/schema.prisma</code>
Seeder	<code>prisma/seed.ts</code>	<code>apps/api/prisma/seed.ts</code>
<code>DB::</code> / Eloquent disponível em todo lugar	<code>PrismaService</code> injetado	<code>apps/api/src/prisma/</code>
<code>routes/api.php</code>	Decorators no controller (<code>@Get</code> , <code>@Post</code>)	<code>*.controller.ts</code>
Controller	Controller (só HTTP)	<code>modules/<feature>/</code>
FormRequest (<code>rules()</code>)	DTO + class-validator	<code>modules/<feature>/dto/</code>
Service class (<code>app/Services</code>)	<code>@Injectable()</code> service	<code>modules/<feature>/*.service.ts</code>
Service Provider / container	Module + injeção de dependência	<code>*.module.ts</code>
<code>abort(404)</code> / <code>ModelNotFoundException</code>	<code>throw new NotFoundException()</code>	nos services
Exception Handler	Exception Filter	<code>apps/api/src/common/filters/</code>
<code>->paginate()</code>	DTO de paginação + <code>skip</code> / <code>take</code>	<code>apps/api/src/common/dto/</code>
<code>DB::transaction()</code>	<code>prisma.\$transaction()</code>	<code>stock-movements.service.ts</code>
<code>DB::select()</code>	<code>prisma.\$queryRaw</code>	<code>dashboard.service.ts</code>
<code>php artisan make:*</code>	<code>nest g *</code>	terminal

Laravel	NestJS + Prisma	Onde fica aqui
<code>php artisan migrate</code>	<code>npx prisma migrate dev</code>	terminal
<code>php artisan tinker</code>	<code>npx prisma studio</code> (visual)	terminal
<code>php artisan route:list</code>	Swagger em <code>/docs</code>	navegador
PHPUnit / Pest	Vitest	<code>*.spec.ts</code>

A maior diferença: no Laravel você escreve a migration e depois o model. No Prisma é o contrário: você escreve o **model** no `schema.prisma` e o Prisma **gera a migration** comparando o schema com o banco.

Front-end

Laravel + React (Inertia ou SPA)	Next.js (App Router)
<code>routes/web.php</code>	Estrutura de pastas em <code>src/app/</code>
<code>resources/js/Pages/Produtos/Index.jsx</code>	<code>src/app/produtos/page.tsx</code>
Layout (Blade <code>@extends</code> ou <code>Layouts/AppLayout.jsx</code>)	<code>src/app/layout.tsx</code>
Rota com parâmetro <code>/produtos/{id}</code>	Pasta <code>src/app/produtos/[id]/</code>
<code>useEffect(() => axios.get(...))</code>	Componente <code>async</code> que faz <code>await</code> direto no servidor
<code>useForm().post()</code> do Inertia / axios no submit	Server Action + <code>useActionState</code>
Validação: FormRequest no back + erros via Inertia	zod na Server Action + class-validator na API
<code>tailwind.config.js</code>	Tailwind v4: tokens em CSS (<code>@theme</code> no <code>globals.css</code>)
Página 404 / erro do Laravel	<code>not-found.tsx</code> e <code>error.tsx</code>

A maior diferença: no React que você conhece, todo componente roda no navegador. No Next.js, **por padrão o componente roda no servidor** (Server Component). Ele pode ser `async` e buscar dados direto, como um controller do Laravel que renderiza uma view. Só os componentes marcados com `"use client"` rodam no navegador, e eles são necessários apenas onde há interação (`useState` , `onClick` , etc.).

PARTE 1: BACK-END (NestJS)

1. Infraestrutura: o banco MySQL

No Laravel você usaria o Sail ou o MySQL local. Aqui o banco sobe com Docker Compose.

Crie `docker-compose.yml` na raiz com o serviço `mysql:8.4`. A porta é 3307 no host, para não brigar com um MySQL que você já tenha instalado.

```
docker compose up -d
```

BASH

O arquivo `docker/mysql/init.sql` dá ao usuário `estoque` permissão de criar bancos. O Prisma precisa disso em desenvolvimento para criar um "shadow database" temporário ao gerar migrations.

2. Dependências da API

```
cd apps/api
```

BASH

```
# ORM (o equivalente ao Eloquent)
```

```
npm install @prisma/client @prisma/adapters-mariadb
```

```
npm install -D prisma tsx
```

```
# Validação (o equivalente às rules() do FormRequest)
```

```
npm install class-validator class-transformer
```

```
# Config, documentação e segurança
```

```
npm install @nestjs/config @nestjs/swagger helmet zod dotenv
```

3. Configuração e `.env`

Crie `apps/api/.env` (e o `.env.example`):

```
PORT=3333
```

ENV

```
DATABASE_URL="mysql://estoque:estoque@localhost:3307/estoque"
```

```
CORS_ORIGIN=http://localhost:3000
```

Crie `src/config/env.schema.ts`: um schema zod que valida essas variáveis. Se faltar alguma, a API nem sobe e o erro diz qual é. No Laravel um `env('X')` faltando só aparece quando o código passa por ali.

Registre no `app.module.ts`:

```
ConfigModule.forRoot({ isGlobal: true, cache: true, validate: validateEnv },
```

TS

Em qualquer service, leia com `config.get('PORT', { infer: true })`, o equivalente ao `config('app.port')`.

4. Models e migrations (Prisma)

4.1 Configuração do Prisma

Crie `prisma.config.ts` na raiz da API. Ele diz ao CLI onde está o schema, onde ficam as migrations, qual comando roda o seed e qual é a URL do banco.

4.2 Escreva os models

Crie `prisma/schema.prisma`. Compare com o Laravel:

```
// Laravel: migration
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name', 80)->unique();
    $table->string('description', 255)->nullable();
    $table->timestamps();
});

// Laravel: model
class Category extends Model {
    public function products() { return $this->hasMany(Product::class); }
}
```

PHP

```
// Prisma: as duas coisas num lugar só
model Category {
    id          Int          @id @default(autoincrement())
    name        String       @unique @db.VarChar(80)
    description String?      @db.VarChar(255)           // ? = nullable
    products    Product[]    // = hasMany
    createdAt   DateTime     @default(now()) @map("created_at")
    updatedAt   DateTime     @updatedAt @map("updated_at")

    @@map("categories") // nome da tabela
}
```

PRISMA

Pontos para notar no schema do projeto:

- `@map` / `@@map`: no banco fica `snake_case` (como no Laravel), no código fica `camelCase`.
- `belongsTo` é declarado com `@relation(fields: [categoryId], references: [id])`.
- `onDelete: Restrict` = `->restrictOnDelete()` da foreign key.
- `Decimal @db.Decimal(10, 2)` para dinheiro, nunca `Float`.
- `enum Unit { UN CX PCT KIT }` vira um `ENUM` no MySQL e um tipo no TypeScript.
- `@@index([categoryId])` = `$table->index('category_id')`.

4.3 Gere a migration

```
npx prisma migrate dev --name init      # = php artisan make:migration + migrate
npx prisma generate                     # gera o client tipado em src/generated/prisma
```

BASH

O SQL gerado fica em `prisma/migrations/<data>_init/migration.sql`. **Abra e leia:** é o `CREATE TABLE` que você escreveria na mão.

Para alterar o banco depois (ex.: adicionar um campo), você **não** cria uma migration nova à mão:

1. Edita o `schema.prisma` (ex.: adiciona `barcode String? @db.VarChar(20)` em `Product`).
2. Roda `npx prisma migrate dev --name add_product_barcode`.
3. O Prisma compara schema x banco e gera só o `ALTER TABLE`.

Laravel	Prisma
<code>php artisan migrate</code>	<code>npx prisma migrate dev</code> (desenvolvimento)
<code>php artisan migrate</code> em produção	<code>npx prisma migrate deploy</code>
<code>php artisan migrate:fresh --seed</code>	<code>npx prisma migrate reset</code>

4.4 O "model" no código: `PrismaService`

No Laravel, `Category::all()` funciona em qualquer lugar. No Nest, o acesso ao banco é um **provider** que o container injeta onde for pedido.

Crie `src/prisma/prisma.service.ts`: uma classe que estende o `PrismaClient` gerado e conecta via adapter do MySQL.

Crie `src/prisma/prisma.module.ts` com `@Global()`. Global significa que, registrado uma vez no `AppModule`, o `PrismaService` fica disponível em todos os módulos sem precisar importar.

Tradução do uso:

```
Category::orderBy('name')->withCount('products')->get();
Category::findOrFail($id);
Category::create($data);
$category->update($data);
$category->delete();
```

PHP

```
this.prisma.category.findMany({ orderBy: { name: 'asc' }, include: { _count: { select: {
products: true } } } });
this.prisma.category.findUnique({ where: { id } }); // + if (!x) throw new
NotFoundException()
this.prisma.category.create({ data });
this.prisma.category.update({ where: { id }, data });
this.prisma.category.delete({ where: { id } });
```

TS

4.5 Seeder

Crie `prisma/seed.ts` e rode `npx prisma db seed` (= `php artisan db:seed`). O comando é o que está em `prisma.config.ts`.

5. Estrutura de pastas da API

No Laravel as pastas são por **tipo** (`Controllers/`, `Models/`, `Requests/`). No Nest a convenção é por **feature**: cada módulo tem tudo o que precisa.

```
apps/api/src/
├─ main.ts                ≈ bootstrap/app.php + public/index.php
├─ app.module.ts          ≈ lista de providers + arquivos de rota
├─ config/                ≈ config/
├─ prisma/                conexão com o banco (provider global)
├─ common/               código compartilhado entre features
│   └─ dto/              paginação
│       └─ filters/      ≈ Exception Handler
│           └─ transforms.ts  helpers de DTO (trim, toBoolean)
└─ modules/
    └─ categories/
        ├── categories.module.ts    ≈ ServiceProvider da feature
        ├── categories.controller.ts ≈ Controller + rotas
        ├── categories.service.ts   ≈ Service / regras
        └─ dto/
            ├── create-category.dto.ts ≈ StoreCategoryRequest
            └─ update-category.dto.ts  ≈ UpdateCategoryRequest
```

6. Criando uma feature completa (Categorias)

A ordem dentro de cada feature é sempre a mesma: **module** → **DTOs** → **service** → **controller** → **registrar** → **testar**.

6.1 Gere os arquivos com o CLI

Como o `php artisan make:controller`:

```
nest g module modules/categories      # cria o module e JÁ importa no AppModule
nest g service modules/categories --no-spec
nest g controller modules/categories --no-spec
```

BASH

Ou tudo de uma vez (≈ `php artisan make:model Category -mcr --requests`):

```
nest g resource modules/categories --no-spec
# pergunta: transport layer → REST API; gerar CRUD → Yes
```

BASH

O `resource` também cria uma pasta `entities/`. Com Prisma ela não é necessária (os tipos vêm do client gerado), pode apagar.

6.2 DTOs = FormRequests

Crie dto/create-category.dto.ts:

```
// Laravel
public function rules() {
    return [
        'name' => ['required', 'string', 'max:80'],
        'description' => ['nullable', 'string', 'max:255'],
    ];
}
```

PHP

```
// Nest: cada regra é um decorator na propriedade
export class CreateCategoryDto {
    @Transform(trim)
    @IsString()
    @IsNotEmpty()
    @MaxLength(80)
    name: string;

    @IsOptional()
    @Transform(trim)
    @IsString()
    @MaxLength(255)
    description?: string;
}
```

TS

Crie o DTO de update reaproveitando o de criação:

```
export class UpdateCategoryDto extends PartialType(CreateCategoryDto) {} // tudo opcional
('sometimes')
```

TS

Quem executa a validação é o `ValidationPipe` global (passo 7). Com ele, um DTO inválido devolve **400** automaticamente, como o FormRequest devolve 422. Regras que o Laravel faz no FormRequest, como `unique:categories`, aqui ficam no banco (índice único) + filtro de exceção (passo 8).

6.3 Service = regras de negócio

Crie categories.service.ts. O `@Injectable()` registra a classe no container. O construtor pede o `PrismaService` e o Nest entrega (como type-hint no construtor do Laravel):

TS

```

@Injectable()
export class CategoriesService {
  constructor(private readonly prisma: PrismaService) {}

  async findOne(id: number) {
    const category = await this.prisma.category.findUnique({ where: { id } });
    if (!category) throw new NotFoundException(`Categoria ${id} não encontrada`); // =
    abort(404)
    return category;
  }

  async remove(id: number) {
    const category = await this.findOne(id);
    if (category._count.products > 0) throw new ConflictException('...'); // = abort(409)
    await this.prisma.category.delete({ where: { id } });
  }
}

```

Regra de ouro: se é regra de negócio (não pode excluir categoria com produtos, saldo não pode ficar negativo), fica no **service**, nunca no controller.

6.4 Controller = rotas + HTTP

Crie `categories.controller.ts`. No Nest **não existe arquivo de rotas**: a rota é declarada no próprio controller com decorators.

PHP

```

// Laravel: routes/api.php
Route::apiResource('categories', CategoryController::class);

```

TS

```

@Controller('categories') // prefixo: /categories
export class CategoriesController {
  constructor(private readonly categories: CategoriesService) {}

  @Get() // GET /categories
  findAll() { return this.categories.findAll(); }

  @Get(':id') // GET /categories/:id
  findOne(@Param('id', ParseIntPipe) id: number) { return this.categories.findOne(id); }

  @Post() // POST /categories
  create(@Body() dto: CreateCategoryDto) { return this.categories.create(dto); }

  @Patch(':id') // PATCH /categories/:id
  update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateCategoryDto) { ... }

  @Delete(':id') // DELETE /categories/:id
  @HttpCode(HttpStatus.NO_CONTENT) // 204
  remove(@Param('id', ParseIntPipe) id: number) { ... }
}

```

- `@Body() dto: CreateCategoryDto` ≈ `store(StoreCategoryRequest $request)`.

- `ParseIntPipe` converte `"5"` em `5` e devolve 400 se não for número.
- O retorno vira JSON automaticamente (como retornar um array no Laravel).

6.5 Module = amarra tudo

Crie/confira `categories.module.ts`:

```
@Module({
  controllers: [CategoriesController],
  providers: [CategoriesService],
})
export class CategoriesModule {}
```

TS

E ele precisa estar no `imports` do `app.module.ts` (o `nest g module` já faz isso). **Se você esquecer esse passo, as rotas não existem**, o equivalente a não carregar o arquivo de rotas.

7. `main.ts`: configuração global

Edite `src/main.ts`. É onde se liga o que vale para a API inteira:

```
app.setGlobalPrefix('api'); // /api/...
app.enableVersioning({ type: VersioningType.URI, defaultVersion: '1' }); // /api/v1/...
app.enableCors({ origin: ... });
app.use(helmet()); // headers de segurança

app.useGlobalPipes(new ValidationPipe({
  whitelist: true, // ignora campos fora do DTO (= $request->validated())
  forbidNonWhitelisted: true, // ...ou melhor, rejeita a requisição
  transform: true, // converte o JSON em instância do DTO
}));

app.useGlobalFilters(new PrismaExceptionFilter()); // passo 8

SwaggerModule.setup('docs', app, ...); // documentação em /docs
```

TS

Nesse ponto rode `npm run start:dev` e abra `http://localhost:3333/docs`: o Swagger lista as rotas (≈ `route:list`) e deixa testar cada uma pelo navegador.

8. Tratamento de erros global

Sem nada, violar o índice único de `name` vira um **500**. No Laravel você trataria no `Handler`. Aqui:

Crie `common/filters/prisma-exception.filter.ts` um `@Catch(Prisma.PrismaClientKnownRequestError)` que converte:

- `P2002` (unique) → **409** "Já existe um registro com este valor em: name"
- `P2025` (não encontrado) → **404**
- `P2003` (foreign key) → **409**

9. Código compartilhado: paginação

Crie `common/dto/pagination-query.dto.ts` com `page` e `pageSize` validados, e o helper `paginate()` que monta `{ data, meta: { page, pageSize, total, totalPages } }` (o formato do `->paginate()` do Laravel, simplificado).

DTOs de listagem **estendem** esse: `list-products-query.dto.ts` adiciona `search`, `categoryId`, `status` e `archived`. No controller:

```
@Get()  
findAll(@Query() query: ListProductsQueryDto) { ... } // query string validada como um body
```

TS

10. Features seguintes

Repita o passo 6 para cada feature. O que cada uma ensina de novo:

Produtos (`modules/products`)

- `OmitType` + `PartialType` no DTO de update: o saldo **não** é editável por PATCH.
- Filtro com `OR` (busca por nome ou SKU) $\approx$ `where(fn($q) => $q->where(...)->orWhere(...))`.
- Comparar duas colunas (`quantity <= min_quantity`) com `this.prisma.product.fields.minQuantity` $\approx$ `whereColumn()`.
- `$transaction([findMany, count])`: página + total numa ida ao banco.
- Criar produto + movimentação de saldo inicial numa transação.
- Exclusão bloqueada se houver histórico → a alternativa é arquivar (`active: false`), um soft delete manual.

Movimentações (`modules/stock-movements`)

A regra de negócio mais importante do sistema:

```
// Laravel  
DB::transaction(function () use ($data) {  
    $product = Product::lockForUpdate()->findOrFail($data['product_id']);  
    // valida saldo, atualiza, cria movimento  
});
```

PHP

```
// Nest + Prisma (stock-movements.service.ts)
return this.prisma.$transaction(async (tx) => {
  const product = await tx.product.findUnique(...);
  // ... calcula delta, valida saldo (422 se ficar negativo)
  const { count } = await tx.product.updateMany({
    where: { id: product.id, quantity: product.quantity }, // só atualiza se ninguém mexeu
    data: { quantity: balanceAfter },
  });
  if (count === 0) throw new ConflictException(...); // alguém mexeu: 409
  return tx.stockMovement.create(...);
});
```

Em vez de `lockForUpdate` (lock pessimista), usa **lock otimista**: o `UPDATE` só acontece se o saldo ainda for o que foi lido. O controller só tem `GET` e `POST`: o histórico é imutável.

Dashboard (`modules/dashboard`)

Agregações com SQL puro (`DB::select`). O template string do `$queryRaw` vira prepared statement, então `${since}` é um parâmetro seguro, não concatenação.

11. Testes

Crie `stock-movements.service.spec.ts`. O service é instanciado com `new` e um Prisma falso: sem banco, roda em milissegundos. É o equivalente a um teste unitário com mock no PHPUnit.

```
npm test
```

Resumo da ordem no back-end

```
docker compose up
→ .env + config/env.schema.ts + ConfigModule
→ schema.prisma → prisma migrate dev → prisma generate
→ PrismaService + PrismaModule (@Global)
→ seed.ts
→ main.ts (prefixo, ValidationPipe, filtros, Swagger)
→ common/ (filtro de erros, paginação)
→ para cada feature:
    nest g module → dto/ → service → controller → (module registrado) → testar no /docs
→ testes do service com regra de negócio
```

PARTE 2: FRONT-END (Next.js)

12. Como o Next organiza as coisas

12.1 Rotas = pastas

Não existe `routes/web.php` nem React Router. **A pasta é a URL**, e o arquivo `page.tsx` dentro dela é a página:

```
src/app/
├─ layout.tsx          layout de TODAS as páginas (sidebar)
├─ page.tsx            /
├─ error.tsx           tela de erro (qualquer rota)
├─ not-found.tsx       tela 404
├─ globals.css         Tailwind + tokens de cor
├─ icon.svg            favicon (convenção de nome)
├─ categorias/
│   └─ page.tsx        /categorias
├─ movimentacoes/
│   └─ page.tsx        /movimentacoes
├─ produtos/
│   └─ page.tsx        /produtos
│   └─ novo/
│       └─ page.tsx    /produtos/novo
│   └─ [id]/
│       └─ page.tsx    parâmetro, ≈ {id} no Laravel
│       └─ load-product.ts  arquivo auxiliar (não vira rota: só page.tsx vira)
│       └─ editar/
│           └─ page.tsx /produtos/5/editar
```

Arquivos com nomes especiais: `page`, `layout`, `loading`, `error`, `not-found`, `route` (API endpoints). Qualquer outro nome dentro de `app/` é só um arquivo comum.

12.2 Onde fica o resto: pastas por feature

Com Inertia você provavelmente tem `Pages/`, `Components/`, `Layouts/`. Aqui `app/` fica **fino** (só rotas) e a lógica mora em `features/`:

```

src/
├─ app/                rotas (páginas finas que só montam as peças)
├─ components/
│   └─ ui/             visuais genéricos: Button, Field, Panel, Skeleton...
│       └─ layout/     sidebar
├─ features/           uma pasta por domínio, espelhando os módulos da API
│   └─ categories/
│   └─ dashboard/
│   └─ movements/
│   └─ products/
│       └─ api.ts      LEITURA: funções que chamam GET na API
│       └─ actions.ts  ESCRITA: Server Actions (POST/PATCH/DELETE)
│       └─ schemas.ts  validação zod dos formulários
│       └─ components/ componentes só desta feature
└─ lib/               infra do front: cliente HTTP, tipos, formatação

```

Regra prática: `components/ui` **não sabe nada de estoque** (um `Button` serve em qualquer projeto).
`features/products/components` **sabe** (uma `ProductTable` só faz sentido aqui).

13. Configuração inicial

13.1 Dependências e env

```

cd apps/web
npm install zod server-only @phosphor-icons/react

```

BASH

Crie `apps/web/.env.local`:

```

API_URL=http://localhost:3333/api/v1

```

ENV

Sem o prefixo `NEXT_PUBLIC_`, a variável **só existe no servidor**. Como o navegador nunca chama a API diretamente (só o servidor do Next), a URL não precisa ser pública.

13.2 `next.config.ts`

```

const nextConfig: NextConfig = { cacheComponents: true };

```

TS

Liga o modelo de cache do Next 16: **nada é cacheado por padrão**, e o que for cacheado é marcado explicitamente (passo 20).

13.3 Tailwind v4 e tokens de cor

No Tailwind v3 você teria um `tailwind.config.js` com `theme.extend.colors`. No v4 **não existe mais esse arquivo**: tudo fica no CSS.

Edite `app/globals.css`:

```
@import "tailwindcss";

:root {
  /* modo claro */
  --panel: #fbfbfa;
  --ink: #16191c;
  --signal: #f2c100;
}

@media (prefers-color-scheme: dark) {
  :root { --panel: #191c1f; --ink: #ecedeb; } /* modo escuro */
}

@theme inline {
  /* = theme.extend.colors */
  --color-panel: var(--panel); /* gera bg-panel, text-panel, border-panel... */
  --color-ink: var(--ink);
  --color-signal: var(--signal);
}
```

Nos componentes você usa `bg-panel text-ink`, e o modo escuro troca sozinho, sem escrever `dark:` em cada classe.

14. Layout raiz

Edite `app/layout.tsx` (≈ `AppLayout.jsx` ou `@extends('layouts.app')`): carrega as fontes com `next/font`, aplica o grid sidebar + conteúdo e renderiza `{children}`, que é a página atual.

A sidebar mostra como separar servidor e cliente:

- `sidebar.tsx`: Server Component (marca e estrutura).
- `nav-list.tsx`: a lista de links, sem hooks.
- `active-nav.tsx`: `"use client"`, e o **único** pedaço que precisa do navegador (`usePathname()` para saber o link ativo).

A ideia: empurrar o `"use client"` para a **menor peça possível**.

15. `lib/`: a base do front

Crie nesta ordem:

1. `lib/types.ts`: os tipos das respostas da API (`Product`, `Category`, `Paginated<T>`). É o contrato entre front e back.
2. `lib/api-client.ts`: um `fetch` com URL base, JSON e erros tratados. É o seu "axios configurado". O `import "server-only"` na primeira linha **quebra o build** se algum componente de navegador tentar importá-lo.
3. `lib/format.ts`: moeda, datas e números em pt-BR com `Intl`, sem biblioteca.
4. `lib/action-state.ts` e `lib/action-helpers.ts`: o formato padrão de resposta dos formulários (`status`, `message`, `fieldErrors`). São dois arquivos porque um é importado no navegador e o outro só no servidor.
5. `lib/use-form-action.ts`: hook que liga um formulário a uma Server Action (passo 19).

16. Componentes de UI

Crie em `components/ui/` as peças visuais reutilizáveis, antes das páginas:

Componente	Para quê
<code>button.tsx</code>	<code>Button</code> , <code>ButtonLink</code> e <code>buttonClass()</code> com variantes
<code>field.tsx</code>	<code>Field</code> (label + input + erro, com acessibilidade), <code>Input</code> , <code>Select</code> , <code>Textarea</code>
<code>submit-button.tsx</code>	botão que se desabilita enquanto o form envia
<code>form-message.tsx</code>	mensagem geral de sucesso ou erro
<code>page-header.tsx</code>	título, descrição e botões de ação da página
<code>panel.tsx</code>	caixa com título
<code>skeleton.tsx</code>	placeholder de carregamento
<code>empty-state.tsx</code>	tela de "nenhum item"
<code>pagination.tsx</code>	paginação por links (a URL é a fonte da verdade)

17. Uma feature no front: leitura (Produtos)

A ordem por feature: `api.ts` → **página de listagem** → **componentes da listagem**.

17.1 `api.ts`: as consultas

Crie `features/products/api.ts`:

```
import "server-only";

export function getProducts(query: ProductQuery = {}) {
  return api<Paginated<Product>>("/products", { query: { pageSize: 15, ...query } });
}
export function getProduct(id: number) {
  return api<Product>(`/products/${id}`);
}
```

TS

17.2 A página: um Server Component `async`

Crie `app/produtos/page.tsx`. Compare com o jeito que você faz hoje:

JSX

```
// React tradicional (roda no navegador)
function Produtos() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => { axios.get('/api/products').then(r => { setData(r.data); setLoading(false); }); }, []);
  if (loading) return <Spinner />;
  return <Table data={data} />;
}
```

TSX

```
// Next.js (roda no servidor)
export default function ProductsPage({ searchParams }: PageProps<"/produtos">) {
  return (
    <>
      <PageHeader title="Produtos" />                                {/* aparece na hora */}
      <Suspense fallback={<TableSkeleton />}>                       {/* placeholder enquanto carrega */}
        <Results searchParams={searchParams} />
      </Suspense>
    </>
  );
}

async function Results({ searchParams }) {
  const params = await searchParams;                                // ?search=tubo&page=2
  const { data, meta } = await getProducts(params);                 // await direto, sem useEffect
  return <ProductTable products={data} />;
}
```

- Sem `useState` / `useEffect` / loading manual: o `<Suspense>` cuida do carregamento.
- `searchParams` é uma **Promise** no Next 16 e precisa de `await`.
- `PageProps<"/produtos">` é um tipo global gerado pelo Next a partir das pastas.

17.3 Componentes da feature

- `product-table.tsx`: Server Component, só HTML.
- `product-filters.tsx`: "use client", porque tem input com debounce. Ele **não guarda os resultados**: escreve na URL (`router.replace('?search=...')`) e o servidor refaz a página. Filtro na URL = link compartilhável, F5 e botão voltar funcionando.

17.4 Rota com parâmetro: `[id]`

Crie `app/produtos/[id]/page.tsx`:

TSX

```
async function ProductDetail({ params }) {
  const { id } = await params;                                       // = Route::get('/produtos/{id}')
  const product = await loadProduct(id);
  ...
}
```

`load-product.ts` chama `notFound()` se a API devolver 404 (`≈ findOrFail`), o que renderiza o `not-found.tsx`.

18. Telas de estado

- `app/error.tsx`: `"use client"` obrigatório. Captura qualquer erro não tratado (ex.: API fora do ar) e mostra um botão "tentar de novo".
- `app/not-found.tsx`: o 404.
- Skeletons nos `fallback` dos `<Suspense>` no lugar de spinners.

19. Uma feature no front: escrita (formulários)

A ordem: `schemas.ts` → `actions.ts` → componente de formulário → página.

19.1 O fluxo completo

```
[form no navegador]
  | submit
  ▼
[Server Action] features/products/actions.ts (roda no servidor do Next)
  | 1. valida com zod (schemas.ts) → erro? devolve fieldErrors para o form
  | 2. chama a API (POST /products) → 409/422? devolve a mensagem para o form
  | 3. invalida cache / refresh()
  | 4. redirect('/produtos/24') ou devolve { status: 'success' }
  ▼
[API NestJS] valida de novo com o DTO (a validação que realmente vale)
```

Com Inertia seria `useForm().post('/produtos')` indo para um controller Laravel. Aqui a Server Action é esse "controller do front". Não existe rota de API escrita por você: o Next cria o endpoint da action sozinho.

19.2 `schemas.ts`

Crie `features/products/schemas.ts` com zod. Campos de `<form>` chegam **sempre como string**, e os helpers em `lib/form-schemas.ts` convertem para número tratando vazio como "não preenchido".

19.3 `actions.ts`

Crie `features/products/actions.ts`:

```

"use server"; // todas as funções exportadas viram Server TS
Actions

export async function createProduct(_prev: ActionState, formData: FormData) {
  const values = formValues(formData);
  const parsed = productSchema.safeParse(values);
  if (!parsed.success) return validationError(parsed.error, values); // = erros do
  FormRequest

  let product: Product;
  try {
    product = await api<Product>("/products", { method: "POST", body: parsed.data });
  } catch (error) {
    return apiError(error, values); // ex.: SKU duplicado
    (409)
  }

  updateTag(CATEGORIES_TAG); // invalida o cache de categorias
  redirect(`/produtos/${product.id}`); // = return redirect()->route(...)
}

```

`redirect()` funciona lançando uma exceção especial, por isso fica **fora** do `try/catch`.

19.4 Componente de formulário

Crie `product-form.tsx` com `"use client"`:

```

export function ProductForm({ categories, product, action }) { TSX
  const { state, pending, formProps } = useFormAction(action);

  return (
    <form {...formProps}>
      <Field label="Nome" error={state.fieldErrors?.name}>
        {(a) => <Input name="name" defaultValue={state.values?.name ?? product?.name} {...a}
      />}
      </Field>
      <FormMessage state={state} /> /* erro da API */
      <SubmitButton pending={pending}>Salvar</SubmitButton>
    </form>
  );
}

```

- Os inputs usam `name` e `defaultValue`, sem `useState` por campo (≈ um form HTML comum).
- O mesmo componente serve para criar e editar. A página decide qual action passar.

19.5 As páginas de formulário

- `app/produtos/novo/page.tsx`: busca as categorias no servidor e passa `action={createProduct}`.
- `app/produtos/[id]/editar/page.tsx`: passa `action={updateProduct.bind(null, product.id)}`. O `bind` "amarra" o id como primeiro argumento da action.

20. Cache

Com `cacheComponents: true`, toda leitura é fresca, a menos que você marque:

```
// features/categories/api.ts
export async function getCategories() {
  "use cache"; // guarda o resultado no servidor
  cacheTag("categories"); // etiqueta para invalidar depois
  cacheLife({ stale: 30, revalidate: 60, expire: 120 });
  return api<Category[]>("/categories");
}
```

TS

Depois de alterar uma categoria, a action chama `updateTag("categories")` (≈ `Cache::forget('categories')`). Dados de estoque **não** são cacheados; as actions de movimentação chamam `refresh()` para a página atual buscar de novo.

Resumo da ordem no front-end

```
.env.local + next.config.ts (cacheComponents)
→ globals.css (tokens) + layout.tsx (fontes, sidebar)
→ lib/ (types → api-client → format → action-state/helpers → use-form-action)
→ components/ui/ (Button, Field, Panel, Skeleton...)
→ error.tsx + not-found.tsx
→ para cada feature:
    LEITURA: features/x/api.ts → app/x/page.tsx (async + Suspense) → components/
    ESCRITA: features/x/schemas.ts → features/x/actions.ts → components/x-form.tsx →
    app/x/novo/page.tsx
```

PARTE 3: RECEITA PARA UMA FEATURE NOVA

Exemplo: **Fornecedores**, do banco até a tela. É o exercício 1 do README.

Back-end (apps/api):

1. `schema.prisma`: `model Supplier { id, name, document @unique, ... }` e a relação em `StockMovement ( supplierId Int? )`.
2. `npx prisma migrate dev --name add_suppliers` e `npx prisma generate`.
3. `nest g resource modules/suppliers --no-spec` (REST, com CRUD) e apague `entities/`.
4. `dto/create-supplier.dto.ts` com os decorators; `update` com `PartialType`.
5. `suppliers.service.ts` usando `this.prisma.supplier`.
6. `suppliers.controller.ts`: ajuste as rotas e coloque `ParseIntPipe` nos `:id`.
7. Confira que `SuppliersModule` está no `AppModule`.
8. Adicione `supplierId?` no `CreateStockMovementDto` e use no service.
9. Teste no Swagger (`/docs`).

Front-end (apps/web):

1. `lib/types.ts`: `interface Supplier`.
2. `features/suppliers/api.ts`: `getSuppliers()`.
3. `features/suppliers/schemas.ts` e `actions.ts`: `saveSupplier`, `deleteSupplier`.
4. `features/suppliers/components/supplier-form.tsx`.
5. `app/fornecedores/page.tsx`: lista + formulário (copie o padrão de `app/categorias/page.tsx`).
6. Adicione o link em `components/layout/nav-list.tsx`.
7. No `movement-form.tsx`, um `<Select name="supplierId">` que aparece só quando o tipo é Entrada.

Se você conseguir fazer isso sem olhar este guia, entendeu a arquitetura.